

Containerizing an Express.js App with Nginx

Docker Compose Deployment — Submission Report

Project	nginx-deployment
Git Repository	https://github.com/Byttie/nginx-deployment
DockerHub Image	https://hub.docker.com/repository/docker/rag1b/my-app/general
Docker Image Tag	rag1b/my-app:1.0
Exposed Port	8080 (host) -> 8080 (nginx container)

Overview

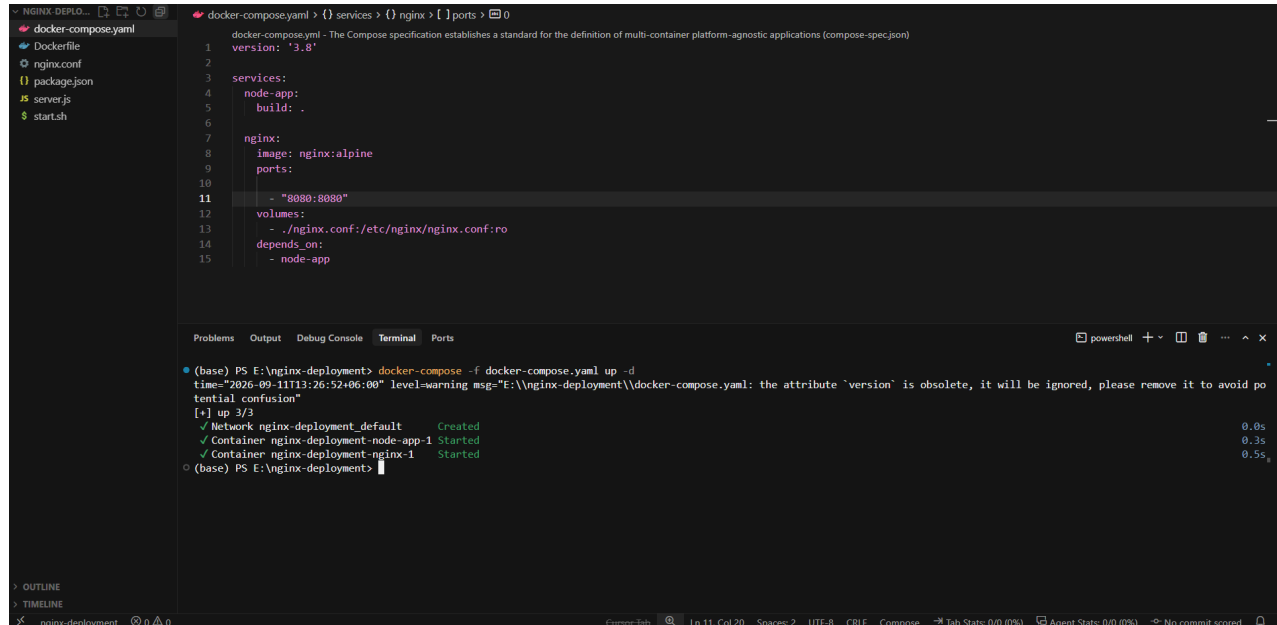
This report documents the steps taken to containerize a Node.js (Express) application and run it together with an Nginx reverse proxy using Docker Compose. The Express app was built into a Docker image, pushed to DockerHub, and orchestrated alongside Nginx so that the application is reachable in the browser on port 8080.

Steps completed:

1. Wrote a Dockerfile for the Node.js/Express application (based on node:18-alpine).
2. Built the Docker image and tagged it as rag1b/my-app:1.0.
3. Pushed the built image to DockerHub (rag1b/my-app).
4. Wrote a docker-compose.yml defining the nginx and node-app services, with nginx depending on node-app and port 8080 exposed to the host.
5. Ran the stack with docker-compose up -d and confirmed both containers started.
6. Verified the deployment by opening localhost:8080 in the browser.

Step 1 — docker-compose.yml and Running the Stack

The docker-compose.yml file defines two services: **node-app** (built from the local Dockerfile) and **nginx** (using the official nginx:alpine image). Nginx maps host port 8080 to container port 8080, mounts a custom nginx.conf, and uses depends_on so that it starts after the node-app service. Running `docker-compose -f docker-compose.yml up -d` in the terminal built and started the network and both containers successfully (Network created, node-app container started, nginx container started).



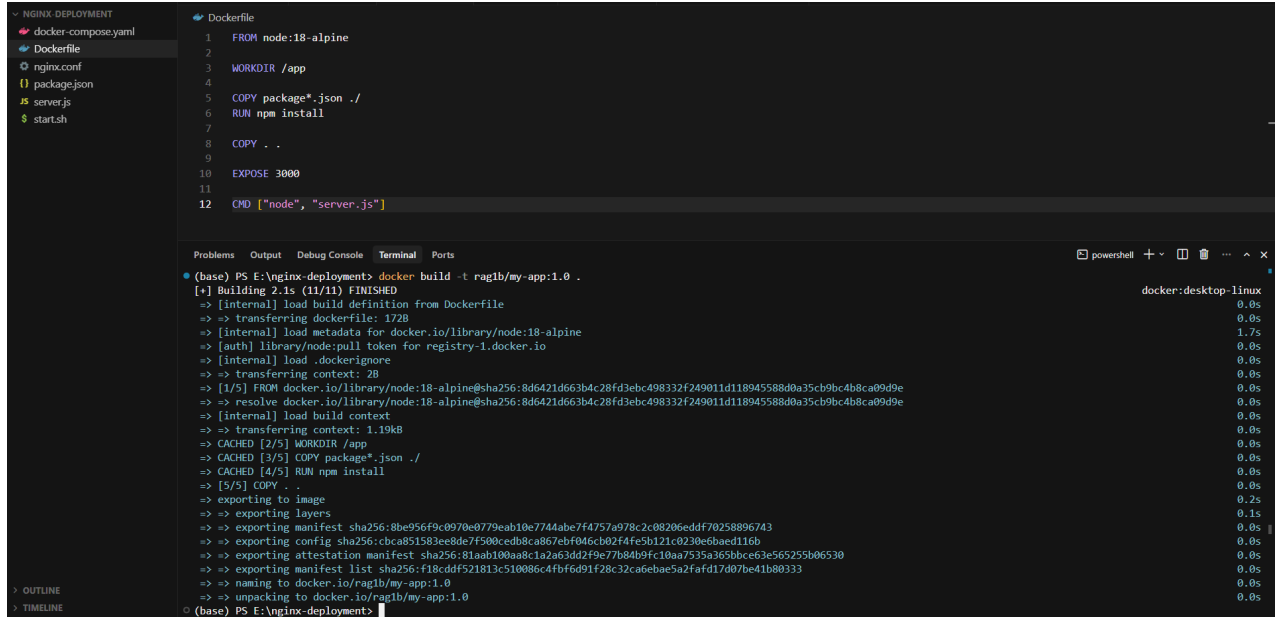
```
docker-compose.yml
1  docker-compose.yml - The Compose specification establishes a standard for the definition of multi-container platform-agnostic applications (compose-spec.json)
2  version: '3.8'
3
4  services:
5    node-app:
6      build: .
7
8    nginx:
9      image: nginx:alpine
10     ports:
11       - "8080:8080"
12     volumes:
13       - ../nginx.conf:/etc/nginx/nginx.conf:ro
14     depends_on:
15       - node-app
```

```
(base) PS E:\nginx-deployment> docker-compose -f docker-compose.yml up -d
time="2026-09-11T13:26:52+06:00" level="warning" msg="E:\nginx-deployment\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 3/3
✓ Network nginx-deployment_default Created 0.0s
✓ Container nginx-deployment-node-app-1 Started 0.3s
✓ Container nginx-deployment-nginx-1 Started 0.5s
(base) PS E:\nginx-deployment>
```

Figure: 1.png

Step 2 — Dockerfile and Building the Express App Image

The Dockerfile for the Express.js app uses `node:18-alpine` as the base image, sets the working directory to `/app`, copies `package.json` and installs dependencies, copies the rest of the source code, exposes port 3000, and starts the app with `node server.js`. The image was built and tagged with `docker build -t rag1b/my-app:1.0 .`, completing all 5 build steps successfully and producing the local image `rag1b/my-app:1.0`.



```

  1 FROM node:18-alpine
  2
  3 WORKDIR /app
  4
  5 COPY package*.json ./
  6 RUN npm install
  7
  8 COPY . .
  9
 10 EXPOSE 3000
 11
 12 CMD ["node", "server.js"]

```

```

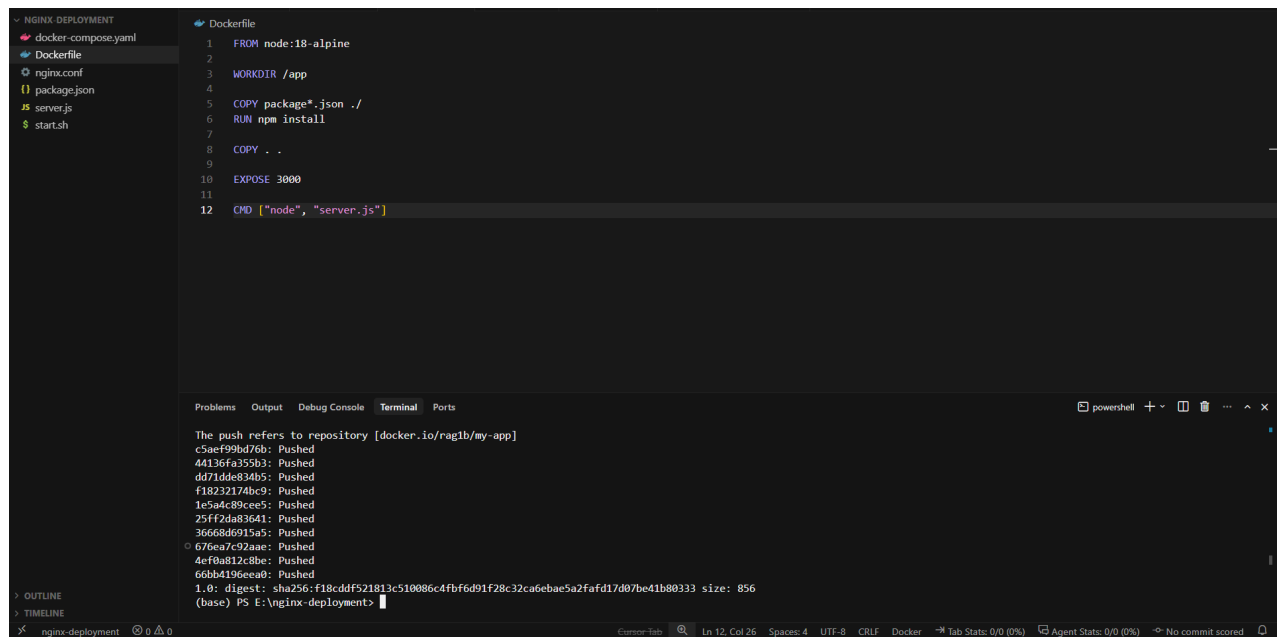
(base) PS E:\nginx-deployment> docker build -t rag1b/my-app:1.0 .
[+] Building 2.1s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> [internal] load dockerfile: 172B
=> [internal] load metadata for docker.io/library/node:18-alpine 1.7s
=> [auth] library/node:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e 0.0s
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 1.19kB 0.0s
=> CACHED [2/5] WORKDIR /app 0.0s
=> CACHED [3/5] COPY package*.json ./ 0.0s
=> CACHED [4/5] RUN npm install 0.0s
=> [5/5] COPY . . 0.0s
=> exporting to image 0.2s
=> => exporting layers 0.1s
=> => exporting manifest sha256:8be956f9c0970e0779eah10e7744abe7f4757a978c2c08206edd70258896743 0.0s
=> => exporting config sha256:cbca851583ee8dc7f500cedb8ca867ebf046cb02f4fe5b121c0230e6baed116b 0.0s
=> => exporting attestation manifest sha256:81aab100aa8ca2a63dd2f9e77b84b9fc10aa7535a365bcbce63e565255b06530 0.0s
=> => exporting manifest list sha256:f18cddf521813c510086c4fbf6d91f28c32ca6bae5a2fafd17d07be41b80333 0.0s
=> => naming to docker.io/rag1b/my-app:1.0 0.0s
=> => unpacking to docker.io/rag1b/my-app:1.0 0.0s
(base) PS E:\nginx-deployment>

```

Figure: 2.png

Step 3 — Pushing the Image to DockerHub

After building the image locally, it was pushed to the DockerHub repository `docker.io/rag1b/my-app` using `docker push rag1b/my-app:1.0`. All image layers were uploaded successfully, and the push completed with the final manifest digest confirming the image (tag 1.0) is now available on DockerHub.



The screenshot shows a VS Code editor with a file explorer on the left containing files for 'NGINX DEPLOYMENT': `docker-compose.yaml`, `Dockerfile`, `nginx.conf`, `package.json`, `server.js`, and `start.sh`. The `Dockerfile` is open in the editor, showing the following content:

```
1 FROM node:18-alpine
2
3 WORKDIR /app
4
5 COPY package*.json ./
6 RUN npm install
7
8 COPY . .
9
10 EXPOSE 3000
11
12 CMD ["node", "server.js"]
```

Below the editor, the 'Terminal' tab is active, displaying the output of a `docker push` command. The output lists the layers being pushed to the repository `docker.io/rag1b/my-app`:

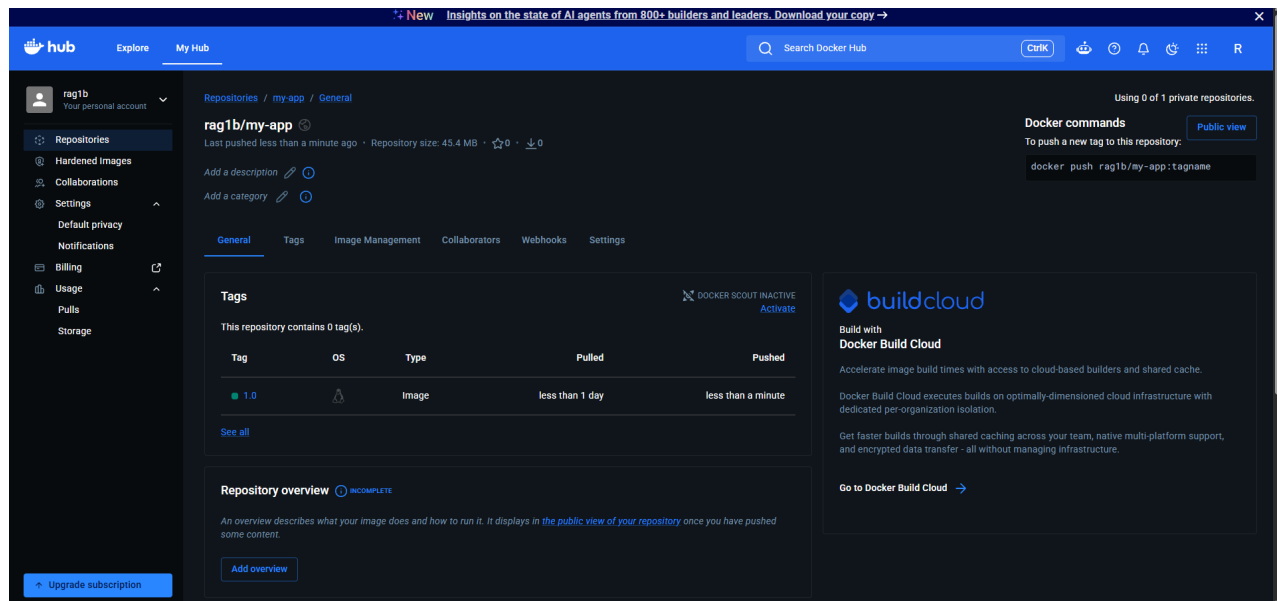
```
The push refers to repository [docker.io/rag1b/my-app]
c5aef99bd76b: Pushed
44136fa355b3: Pushed
dd71dde834b5: Pushed
f18232174bc9: Pushed
1e5a4c89cee5: Pushed
25ff2da83641: Pushed
36668d6915a5: Pushed
676ea7c92aae: Pushed
4ef8a812e8be: Pushed
66bb4196eeab: Pushed
1.0: digest: sha256:f18cddf521813c510086c4fbf6d91f28c32ca6ebae5a2faFd17d07be41b80333 size: 856
(base) PS E:\nginx-deployment>
```

The status bar at the bottom indicates the current file is `nginx-deployment`, line 12, column 26, with 4 spaces, UTF-8 encoding, and CRLF line endings.

Figure: 3.png

Step 4 — Verifying the Image on DockerHub

The DockerHub repository page for **rag1b/my-app** confirms the image was pushed successfully. The Tags tab shows tag **1.0**, pushed less than a minute prior, with a repository size of 45.4 MB, confirming the image is publicly hosted and available for pull.



The screenshot displays the DockerHub interface for the repository **rag1b/my-app**. The left sidebar shows the user's profile and navigation options. The main content area shows the repository details, including the tag **1.0** pushed less than a minute ago. The repository size is 45.4 MB. The Tags tab is active, showing a table with the tag **1.0** and its push time. The repository overview section is also visible.

Tag	OS	Type	Pulled	Pushed
1.0	linux	Image	less than 1 day	less than a minute

Figure: 4.png

Step 5 — Verifying the App in the Browser

With the Docker Compose stack running, the application was verified by navigating to `http://localhost:8080` in the browser. The page successfully displays **"Hello World!"**, confirming that requests reach the Express.js app through the Nginx reverse proxy on the exposed port 8080, completing the deployment.

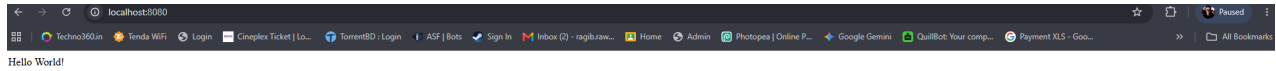


Figure: 5.png

Conclusion

The Express.js application was successfully containerized, pushed to DockerHub, and deployed alongside Nginx using Docker Compose. The stack is exposed on port 8080 and was verified to be working correctly in the browser. Source files (Dockerfile and docker-compose.yml) are available in the linked GitHub repository, and the built image is publicly available on DockerHub at the link listed on the cover page.